

Final Capstone Project

Enterprise GenAI Capstone Project

Enterprise Financial Intelligence Assistant using Retrieval-Augmented Generation (RAG)

Background

FinVista Capital is a financial consulting organization that provides investment research, risk analysis, and market intelligence to corporate clients. The organization stores thousands of financial reports, annual reports, analyst reports, compliance manuals, investment guidelines, and market research documents. Employees spend considerable time searching for relevant information before preparing reports. The organization plans to build an AI-powered Enterprise Financial Intelligence Assistant that enables users to interact with enterprise knowledge through natural language conversations. The primary emphasis of this project is the design, development, and deployment of a production-ready Retrieval-Augmented Generation (RAG) application.

Business Objectives

- Upload enterprise documents
- Automatically process and index documents
- Search across multiple documents using semantic search
- Ask questions in natural language
- Receive context-aware AI-generated responses
- Display source citations
- Support multi-turn conversations
- Reduce manual document search effort by at least 70%

Project Objective

Design, develop, and deploy an Enterprise Financial Intelligence Assistant using Retrieval-Augmented Generation (RAG).

- Ingest enterprise documents
- Process PDF documents
- Generate embeddings
- Store embeddings in a vector database
- Retrieve relevant document chunks
- Generate grounded responses using a Large Language Model
- Display document citations
- Maintain conversation history
- Provide a Streamlit-based web application

Project Scope

Phase 1 – AI Application Development

This phase constitutes the majority of the project effort.

- Document upload
- PDF parsing
- Text preprocessing
- Document chunking

- Embedding generation
- Vector database indexing
- Semantic search
- Context retrieval
- Prompt construction
- AI response generation
- Source citations
- Conversation memory
- Error handling and logging

Phase 2 – User Interface

- Develop a Streamlit application
- Upload documents
- Ask questions
- Display AI responses
- Display source citations
- Review conversation history

Phase 3 – Containerization & Kubernetes Deployment

- Containerize using Docker
- Deploy on Kubernetes (EKS)
- Deployment manifests
- Services
- ConfigMaps and Secrets
- Health checks
- Horizontal Pod Autoscaler (optional)

Phase 4 – CI/CD Pipeline

- Build application
- Build Docker image
- Push image to container registry
- Deploy to Kubernetes
- Support application updates

Functional Requirements

- Enterprise document upload
- Automatic document processing
- Embedding generation
- Vector database storage
- Semantic search
- Retrieval-Augmented Generation
- Context-aware responses
- Source citation
- Conversation history
- Logging
- Error handling

Non-Functional Requirements

- Scalable
- Highly Available
- Secure
- Maintainable
- Containerized
- Cloud Ready
- Production Deployable

Suggested Technology Stack

Programming Language: Python

User Interface: Streamlit

AI Components: Large Language Model, Embedding Model, Vector Database

Deployment: Docker, Kubernetes (EKS or kubeadm setup)

DevOps: Git, CI/CD Pipeline

Deliverables

- Complete Source Code
- Streamlit Application
- Dockerfile
- Kubernetes Deployment Configuration
- CI/CD Pipeline
- Architecture Diagram
- README
- Demonstration Video

Project Evaluation Focus

- End-to-end RAG implementation
- Retrieval quality
- Grounded AI responses with citations
- Overall AI application design
- Kubernetes deployment
- CI/CD automation